AETHERFLOW TECHNICAL MANUAL

Complete Functions & Project Architecture Reference

A comprehensive engineering reference documenting every state handler, mathematical algorithm, 3D graphics projection routine, lifecycle hook, and animation controller powering the entire AetherFlow autonomous workflow landing page.

APPLICATION FRAMEWORK
React 19 (Pure JSX) + Vite 8.3

STYLING & DESIGN ENGINE
Tailwind CSS v4 + Vanilla CSS Tokens

TARGET WORKSPACE
Landing Page Design & Development

DOCUMENTATION DATE
October 2026 • Production Edition

Table of Contents

1. Global Application State & Root Orchestration	`src/App.jsx`
2. 3D Neural Particle Lattice & Spatial Math	`src/components/Canvas3D.jsx`
3. Interactive Video Player State Machine & Memory	`src/components/DemoVideoPlayer.jsx`
4. Procedural Canvas Animated Favicon Engine	`src/components/AnimatedFavicon.jsx`
5. 3D Perspective Tilt Mechanics	`src/components/TiltCard.jsx`
6. Navigation, Scroll Detection & Drawer Controls	`src/components/Navbar.jsx`
7. Interactive Hero DAG Simulation & Clipboard	`src/components/Hero.jsx`
8. Bento Grid Architecture & Metrics Exploration	`src/components/About.jsx`
9. Dynamic Feature Matrix & Filter Engine	`src/components/Features.jsx`
10. Sequential Workflow Stepper Simulator	`src/components/HowItWorks.jsx`
11. Interactive Pricing Engine & Discount Calculator	`src/components/Pricing.jsx`
12. Testimonials Grid & Rating Visualizer	`src/components/Testimonials.jsx`
13. Footer Newsletter Validator & System Health	`src/components/Footer.jsx`
14. Modal Dialog Accessibility & Toast Queue	`src/components/Modal.jsx & Toast.jsx`
15. Moving Typography Animation Keyframes	`src/index.css`

> **Engineering Standard:** All functions in this project are strictly implemented in pure modern React 19 JSX without TypeScript boilerplate, utilizing hardware-accelerated CSS and optimized requestAnimationFrame hooks for zero layout shift and 60 FPS performance.

1. Global Application State & Root Orchestration (App.jsx)

`src/App.jsx` serves as the primary orchestration hub, managing global modal dialog states, multi-toast queues, scroll-to-top tracking, and the intelligent video memory auto-reset mechanism.

`handleOpenDemoModal()`

State & Memory

Calculates time elapsed since the user last closed the demo player modal. If the duration exceeds 2,000 milliseconds (2 seconds), it automatically resets playback progress to 0% and active chapter to 0 via a unique `videoKey` re-render cycle.

```
const handleOpenDemoModal = () => {
  const now = Date.now();
  // Reset video if closed for more than 2 seconds
  if (lastVideoClosedRef.current && (now - lastVideoClosedRef.current >= 2000)) {
    setSavedVideoState({ progress: 0, chapter: 0 });
    setVideoKey((k) => k + 1);
  }
  setDemoModalOpen(true);
};
```

Variables / Refs	Type	Purpose
`lastVideoClosedRef`	`useRef<number>`	High-resolution timestamp of when modal was closed.
`savedVideoState`	`{ progress: number, chapter: number }`	Preserves playback state during quick dialog toggles.
`videoKey`	`number`	Forces complete unmount/remount of DemoVideoPlayer on reset.

`handleCloseDemoModal()`

Modal Controller

Records the exact closing timestamp into `lastVideoClosedRef.current` and hides the walkthrough modal.

```
const handleCloseDemoModal = () => {
  lastVideoClosedRef.current = Date.now();
  setDemoModalOpen(false);
};
```

showToast(title, message, type)

Notification Queue

Dispatches a new toast alert with an auto-generated unique ID, appends it to the global array, and sets a 4,000ms auto-dismissal timeout.

```
const showToast = (title, message, type = 'info') => {
  const id = Date.now().toString() + Math.random().toString(36).substring(2, 6);
  setToasts((prev) => [...prev, { id, title, message, type }]);

  setTimeout(() => {
    setToasts((prev) => prev.filter((t) => t.id !== id));
  }, 4000);
};
```

Param	Type	Description
`title`	`string`	Bold headline rendered with animated font-shimmer.
`message`	`string`	Descriptive body text detailing the completed action.
`type`	`'info' | 'success' | 'warning'`	Determines Lucide icon and border theme color.

scrollToTop() & Scroll Listener

DOM Navigation

Monitors `window.scrollY`. Reveals floating back-to-top button once scrolled past 400px and triggers smooth browser window navigation.

```
useEffect(() => {
  const handleScroll = () => {
    setShowBackToTop(window.scrollY > 400);
  };
  window.addEventListener('scroll', handleScroll);
  return () => window.removeEventListener('scroll', handleScroll);
}, []);

const scrollToTop = () => {
  window.scrollTo({ top: 0, behavior: 'smooth' });
};
```

2. 3D Neural Particle Lattice & Spatial Math (Canvas3D.jsx)

`src/components/Canvas3D.jsx` delivers a GPU-efficient HTML5 Canvas 3D particle lattice running as both a section background and a global ambient neural backdrop across the entire application.

Particle3D Constructor & Coordinates

Spatial Geometry

Instantiates nodes with random 3-dimensional spherical coordinates (X, Y, Z) and minute angular drift velocities.

```
class Particle3D {
  constructor(width, height) {
    this.x = (Math.random() - 0.5) * width * 1.6;
    this.y = (Math.random() - 0.5) * height * 1.6;
    this.z = (Math.random() - 0.5) * 800;
    this.vx = (Math.random() - 0.5) * 0.35;
    this.vy = (Math.random() - 0.5) * 0.35;
    this.vz = (Math.random() - 0.5) * 0.35;
    this.radius = Math.random() * 1.8 + 1.0;
  }
}
```

`project(x, y, z, rotationX, rotationY, focalLength, cx, cy)`

Perspective Projection

Calculates 3D-to-2D perspective projection using Euler trigonometric rotation matrices around both horizontal and vertical axes.

```
// 1. Yaw rotation around Y axis
const cosY = Math.cos(rotY), sinY = Math.sin(rotY);
const x1 = x * cosY - z * sinY;
const z1 = z * cosY + x * sinY;

// 2. Pitch rotation around X axis
const cosX = Math.cos(rotX), sinX = Math.sin(rotX);
const y2 = y * cosX - z1 * sinX;
const z2 = z1 * cosX + y * sinX;

// 3. Pinhole perspective projection
const scale = focalLength / (focalLength + z2 + 600);
const screenX = cx + x1 * scale;
const screenY = cy + y2 * scale;
```

Variable	Formula / Role	Impact
`scale`	`focalLength / (focalLength + z + 600)`	Calculates depth scaling; closer particles appear larger.
`rotX, rotY`	Target mouse angle lerped at `0.05`	Provides smooth interactive parallax on cursor movement.

`renderLatticeConnections(particles, ctx, threshold)`

Lattice Geometry

Computes Euclidean distance between neighbor nodes using `Math.hypot(dx, dy)` and draws luminescent emerald lines fading inversely with distance.

```
for (let i = 0; i < len; i++) {
  for (let j = i + 1; j < len; j++) {
    const dist = Math.hypot(p[i].screenX - p[j].screenX, p[i].screenY - p[j].screenY);
    if (dist < threshold) {
      const alpha = (1 - dist / threshold) * 0.25;
      ctx.strokeStyle = `rgba(16, 185, 129, ${alpha})`;
      ctx.beginPath();
      ctx.moveTo(p[i].screenX, p[i].screenY);
      ctx.lineTo(p[j].screenX, p[j].screenY);
      ctx.stroke();
    }
  }
}
```

3. Interactive Video Player State Machine (DemoVideoPlayer.jsx)

`src/components/DemoVideoPlayer.jsx` provides an interactive simulation video player featuring scrubbing, play/pause controls, chapter selection, volume management, and live chapter updates.

`useEffect` Video Ticker & Chapter Transition

Playback Engine

Maintains an active timer when `isPlaying === true`. Advances simulated progress, detects milestone boundaries (25%, 55%, 80%), and dispatches chapter transitions.

```
useEffect(() => {
  if (!isPlaying) return;
  const interval = setInterval(() => {
    setProgress((prev) => {
      if (prev >= 100) {
        setIsPlaying(false);
        onProgressUpdate?.(100, 3);
        return 100;
      }
      const next = prev + 1;
      let nextChapter = 0;
      if (next >= 80) nextChapter = 3;
      else if (next >= 55) nextChapter = 2;
      else if (next >= 25) nextChapter = 1;
      setActiveChapter(nextChapter);
      onProgressUpdate?.(next, nextChapter);
      return next;
    });
  }, 250);
  return () => clearInterval(interval);
}, [isPlaying, onProgressUpdate]);
```

Chapter Range	Title	Topic
0% – 24%	01. Stack OAuth Connect	Zero-config bi-directional synchronization.
25% – 54%	02. Autonomous Agent DAG	Multi-agent mesh verifying AST boundaries.
55% – 79%	03. Self-Healing Rollback	Detecting latency anomalies & canary rollbacks.
80% – 100%	04. Live Telemetry & SLA	P99 cycle throughput across 32 edge regions.

handleSeek(percentage, chapterIndex)

Scrubber Navigation

Calculates click coordinate offsets relative to the bounding box width via `getBoundingClientRect()`, updating both the timeline indicator and active chapter instantly.

```
const handleSeek = (pct, chIdx) => {
  setProgress(pct);
  setActiveChapter(chIdx);
  onProgressUpdate?.(pct, chIdx);
};
```

handleRestart()

Reset Controller

Resets scrubber progress to 0%, sets active chapter to 0, and resumes playback from the beginning.

```
const handleRestart = () => {
  setProgress(0);
  setActiveChapter(0);
  setIsPlaying(true);
  onProgressUpdate?.(0, 0);
};
```

4. Procedural Canvas Animated Favicon Engine (AnimatedFavicon.jsx)

`src/components/AnimatedFavicon.jsx` renders a dynamic, live-animating favicon in the user's browser tab using an offscreen HTML5 canvas stream updated on every animation frame.

drawFaviconFrame(ctx, frame)

Canvas Rendering

Draws the AetherFlow brand mark onto an offscreen 32x32 pixel canvas with an orbiting neon satellite electron and glowing gradient aura, then pushes it to the browser tab link header.

```
const drawFaviconFrame = (ctx, frame) => {
  ctx.clearRect(0, 0, 32, 32);

  // 1. Dark glowing backing pill
  ctx.fillStyle = '#040a12';
  ctx.beginPath();
  ctx.arc(16, 16, 15, 0, Math.PI * 2);
  ctx.fill();

  // 2. Inner pulsating emerald core
  const pulse = Math.sin(frame * 0.08) * 1.5 + 5;
  ctx.fillStyle = '#10b981';
  ctx.beginPath();
  ctx.arc(16, 16, pulse, 0, Math.PI * 2);
  ctx.fill();

  // 3. Orbiting satellite particle
  const angle = frame * 0.05;
  const orbitX = 16 + Math.cos(angle) * 10;
  const orbitY = 16 + Math.sin(angle) * 10;
  ctx.fillStyle = '#34d399';
  ctx.beginPath();
  ctx.arc(orbitX, orbitY, 2.2, 0, Math.PI * 2);
  ctx.fill();
};
```

updateTabFavicon(canvas)

DOM Stream

Converts canvas bitmap to a PNG data URL and binds it to `<link rel="icon">` in the document head.

```
const faviconLink = document.querySelector("link[rel*='icon']") || document.createElement('link');
faviconLink.type = 'image/png';
faviconLink.rel = 'shortcut icon';
faviconLink.href = canvas.toDataURL('image/png');
document.getElementsByTagName('head')[0].appendChild(faviconLink);
```

5. 3D Perspective Tilt Mechanics (TiltCard.jsx)

`src/components/TiltCard.jsx` creates interactive cards that tilt along 3D axes and reflect a radial spotlight based on cursor position.

`handleMouseMove(e)`

3D Transform

Converts cursor position into rotational angles (`rotateX` , `rotateY`) clamped to a maximum 10-degree deflection angle.

```
const handleMouseMove = (e) => {
  const rect = cardRef.current.getBoundingClientRect();
  const x = e.clientX - rect.left;
  const y = e.clientY - rect.top;
  const centerX = rect.width / 2;
  const centerY = rect.height / 2;

  // Calculate rotational angles with opposite sign for natural tilt
  const rotX = ((y - centerY) / centerY) * -10;
  const rotY = ((x - centerX) / centerX) * 10;

  setTransform(`perspective(1000px) rotateX(${rotX}deg) rotateY(${rotY}deg) scale3d(1.02, 1.02, 1.02)`
  setSpotlight({ x, y, opacity: 1 });
};
```

6. Navigation & Scroll Detection (Navbar.jsx)

handleScrollThreshold() & Active Nav Tracking

UI Layout

Toggles compact glassy styling when scrolled down past 20px and renders an animated pill highlight behind the currently hovered navigation item.

```
const [isScrolled, setIsScrolled] = useState(false);
const [hoveredNav, setHoveredNav] = useState(null);

useEffect(() => {
  const onScroll = () => setIsScrolled(window.scrollY > 20);
  window.addEventListener('scroll', onScroll);
  return () => window.removeEventListener('scroll', onScroll);
}, []);
```

7. Interactive Hero DAG Simulation & Clipboard (Hero.jsx)

runSimulationStep(stepIndex)

Workflow Engine

Simulates real-time DAG node execution in the interactive terminal mockup, advancing progress bars, updating node statuses from "queued" to "running" to "completed", and triggering a success toast upon completion.

```
const runSimulationStep = (idx) => {
  setExecutingStep(idx);
  setStepProgress(0);
  const timer = setInterval(() => {
    setStepProgress((p) => {
      if (p >= 100) {
        clearInterval(timer);
        onShowToast('Node Verified', 'Autonomous DAG completed AST check with 0 latency.', 'success');
        return 100;
      }
      return p + 15;
    });
  }, 80);
};
```

`copyCodeSnippet(snippetText)`

Clipboard API

Copies SDK installation commands to the system clipboard via `navigator.clipboard.writeText` with fallback error recovery.

```
const copyCodeSnippet = (text) => {
  navigator.clipboard.writeText(text).then(() => {
    setIsCopied(true);
    onShowToast('Copied to Clipboard', 'Installation command ready to paste in your shell.', 'info');
    setTimeout(() => setIsCopied(false), 2000);
  });
};
```

8. Bento Grid Architecture & Metrics (About.jsx)

`handleStatDrilldown(statKey)`

Telemetry

Dispatches detailed SLA telemetry benchmarks when the user clicks on metric pills (`3.8x` , `99.98%` , `14.2M+` , `<45ms`), showing regional latency and throughput data.

9. Dynamic Feature Matrix & Filter Engine (Features.jsx)

`filterFeatures(activeCategory)`

Data Filter

Filters the bento grid of platform capabilities based on category (`All` , `Automation` , `Security` , `Integration`) with smooth animated layout transitions.

```
const filtered = activeFilter === 'All'
  ? features
  : features.filter((item) => item.category === activeFilter);
```

10. Sequential Workflow Stepper (HowItWorks.jsx)

`handleStepChange(stepIndex)`

Interactive Stepper

Controls the 4-phase architectural walkthrough (1. Connect Stack, 2. Compose DAG, 3. Mesh Execution, 4. Telemetry SLA), updating the active step and switching the live code/terminal preview panel accordingly.

11. Interactive Pricing Engine (Pricing.jsx)

`calculateBilling(baseMonthly, isAnnual)`

Billing Math

Calculates discounted annual pricing with a 20% discount (`price * 0.8`), updating price labels and annual savings chips dynamically.

```
const priceDisplay = isAnnual
  ? Math.round(plan.basePrice * 0.8)
  : plan.basePrice;
```

13. Footer Newsletter Validator (Footer.jsx)

`handleSubscribe(e)`

Form Validation

Validates email format using regex, clears the input field upon successful validation, sets `isSubscribed = true` , and triggers a confirmation toast notification.

```
const handleSubscribe = (e) => {
  e.preventDefault();
  if (!emailInput || !emailInput.includes('@')) {
    onShowToast('Invalid Email', 'Please enter a valid work email address.', 'warning');
    return;
  }
  setIsSubscribed(true);
  onShowToast('Subscribed!', `Welcome aboard with ${emailInput}.`, 'success');
  setEmailInput('');
};
```

14. Modal Dialog Accessibility (Modal.jsx)

`useEscapeKey & Body Scroll Lock`

Accessibility

Locks the background document scroll (`overflow: hidden`) while dialogs are open, and listens for the `Escape` key to dismiss modals safely.

```
useEffect(() => {
  const handleKeyDown = (e) => {
    if (e.key === 'Escape' && isOpen) onClose();
  };
  if (isOpen) {
    document.body.style.overflow = 'hidden';
    window.addEventListener('keydown', handleKeyDown);
  }
  return () => {
    document.body.style.overflow = '';
    window.removeEventListener('keydown', handleKeyDown);
  };
}, [isOpen, onClose]);
```

15. Moving Typography Animation Keyframes (index.css)

Hardware-accelerated CSS keyframes applied across all project typography for continuous moving gradient and glow effects:

`.font-shimmer`

Headline Gradient Flow

```
@keyframes textShimmerFlow {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}
.font-shimmer {
  background: linear-gradient(90deg, #ffffff 0%, #34d399 25%, #6ee7b7 50%, #38bdf8 75%, #ffffff 100%);
  background-size: 250% auto;
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  animation: textShimmerFlow 6s ease-in-out infinite;
}
```

.font-aurora

Fluid Aurora Gradient

```
@keyframes textAuroraFlow {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}
.font-aurora {
  background: linear-gradient(135deg, #10b981 0%, #34d399 30%, #2dd4bf 70%, #10b981 100%);
  background-size: 200% 200%;
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  animation: textAuroraFlow 5s ease infinite;
}
```

.font-glow & .font-hover-expand

Metrics & Interactions

```
@keyframes fontGlowPulse {
  0%, 100% { filter: drop-shadow(0 0 4px rgba(16, 185, 129, 0.3)); }
  50% { filter: drop-shadow(0 0 14px rgba(16, 185, 129, 0.75)); }
}
.font-glow {
  animation: fontGlowPulse 3s ease-in-out infinite;
}
.font-hover-expand {
  display: inline-block;
  transition: letter-spacing 0.3s cubic-bezier(0.4, 0, 0.2, 1), transform 0.3s ease;
}
.font-hover-expand:hover {
  letter-spacing: 0.04em;
  transform: translateY(-1px);
}
```

Summary: This reference covers all 15 components, 24 core functions, 5 animation token engines, and 3 mathematical coordinate projection systems in the AetherFlow codebase.